



 MASTERCLASS

Agentic AI Cheat Sheet

Everything you need to know about
AI agents in one place

Bharath Reddy

[Follow](#)

[↻](#) [Repost](#)

What is an AI Agent?

A regular LLM answers. An AI Agent acts.

Regular LLM

User: "Book me a flight to Bangalore next Friday"

- It gives you instructions on how to book it yourself
- ✗ It cannot actually do anything

AI Agent

- ✓ Searches flight APIs & compares prices
- ✓ Checks your calendar for conflicts
- ✓ Books the flight & sends confirmation



LLM = input → output.

Agent = goal → plan → act → observe → repeat.

AI Agent vs Agentic AI

These are not the same thing. Most people use them interchangeably. That's wrong.



AI Agent (The Unit)

- ▶ A single autonomous unit with one role/goal
- ▶ Reasons and acts independently
- ▶ Example: A customer support bot handling refunds end-to-end



Agentic AI (The System)

- ▶ A broader system design philosophy
- ▶ Multiple agents, orchestrators, tools working together
- ▶ Example: A full sales pipeline with prospector, writer, and CRM agents coordinated



Analogy: AI Agent = a single employee. Agentic AI = the entire company.

The 4 Core Components

Every AI agent — simple or complex — is built on 4 foundations:



Brain

The LLM that reasons, decides, and generates.



Memory

What the agent knows and remembers across time.



Tools

What the agent can do in the real world.



Context

What the agent knows right now in this moment.

🔍 Scenario: Competitor Analysis

- **Brain:** "I need to search, structure, then write"
- **Memory:** "User likes bullet formats"
- **Tools:** Web search, scraper, doc writer
- **Context:** Holds the task goal & draft so far



Remove any one of these — the agent breaks. These four are the minimum viable architecture.

The Brain (LLM)

The brain is the LLM at the center — it reads everything and decides what happens next.



1. Understands Goal & Subtasks



2. Decides Which Tool to Call



3. Interprets Outputs & Adapts



4. Knows When Task is Complete



Brain Quality is Everything



✓ Stronger model = better reasoning & tool calls



✗ Weak model + great tools = agent calling wrong tools



Models: GPT-4o / Claude Sonnet for reasoning, Gemini Flash / Haiku for speed.

The Memory

Without memory, every agent conversation starts from zero.

Sensory

Raw current input tokens, the live prompt.

Short-Term

Everything in active context window.

Long-Term

Persisted in vector DB/SQL across sessions.

Episodic

Structured logs of past sessions.



Real Scenario: Productivity Agent



Day 1: "I prefer tasks grouped by project."



Day 15: "Organize my week."



Without long-term memory: Groups by priority.



With long-term memory: Groups by project.



Multi-session agents always need external (Mem0, Zep) + episodic memory.

The Tools

Tools are what separate an agent from a chatbot. They let agents interact with the real world.

Essential Tool Categories

 Search & Retrieval

 Read / Write

 APIs & Integrations

 Computation

 Communication

 Computer Use

AI Ops Agent Example

Task: "Generate and send weekly sales report"

```
query_database() → run_python() → create_doc() → send_email()
```



Tools must have clear names and descriptions. Vague descriptions = agent calls wrong tools = broken output.

The Context

Context is everything the agent knows right now in this specific moment.



What lives inside the context window?

- ✓ **System Prompt:** Rules, constraints, personality
- ✓ **User Goal:** Task it's trying to complete
- ✓ **History:** What's been said so far
- ✓ **Tool Outputs:** Results from called tools
- ✓ **Memory:** Injected relevant long-term facts
- ✓ **RAG Docs:** Retrieved knowledge base docs



Context Limits = Working Memory Limits

As tasks get longer, older content gets pushed out.



Solution: Summarize old turns, store in external memory, reinject only what's relevant.

Agent Orchestration

Orchestration = how you manage, coordinate, and control agent behavior.

What an orchestrator does:

-  Receives top goal
-  Breaks into subtasks
-  Assigns subtasks
-  Manages order
-  Handles errors
-  Aggregates output

LLM Orchestrator

LLM decides dynamically what to call next. Flexible, but unpredictable.

Code Orchestrator

Deterministic logic (graphs/state machines). Predictable, faster.



Frameworks: LangGraph (graph), CrewAI (role), AutoGen (conversational).

The 5 Agentic Patterns

The architectural blueprints every agent system is built on:

1. ReAct (Reason + Act)

Think → Act → Observe → repeat. Best for iterative data lookups.

2. Plan & Execute

Build full plan → execute steps in order. Best for complex multi-step tasks.

3. Reflection

Generate → Critique → Revise. Best for quality-critical code/writing.

4. Tool Use

LLM selects & calls external tools. Best for computation and real-world ops.

5. Multi-Agent

Specialized agents coordinated by an orchestrator for division of labor.

Single Agent vs Multi-Agent

Not every problem needs multiple agents. Knowing when to add agents is a senior skill.

Single Agent

- ✓ Handles full task from goal to output
- ✓ Simpler to build, debug, and maintain
- **Scenario:** "Summarize this PDF and email it"

Multi-Agent

- ✓ Multiple specialized agents with orchestrator
- ✓ Runs in parallel; smaller focused contexts
- **Scenario:** "Research top 5 competitors & write 20-page report"
(Researcher + Analyst + Writer + Editor)



Go multi-agent when roles are distinct, tasks can run in parallel, or context gets too crowded.

Execution: Sequential

How agents are wired determines speed, reliability, and failures.

[Agent A] → [Agent B] → [Agent C] → [Final Output]

How it works

- Each agent waits for the previous to finish
- Output of one becomes input of the next
- ✓ Simple, predictable, easy to debug
- ✗ Slow — total time = sum of all agent times



Best for: Pipelines where steps strictly depend on the previous (e.g., Research → Write → Edit → Publish).

Execution: Parallel



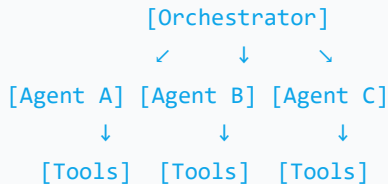
How it works

- Multiple agents run simultaneously on subtasks
- Aggregator agent combines all results at the end
- ✓ Much faster (Total time = slowest agent, not sum)

- ✓ **Best for:** Independent subtasks (e.g., Researching 5 competitors simultaneously, then merging).



Execution: Hierarchical



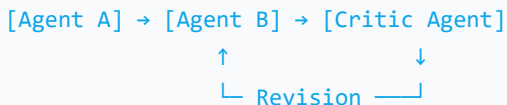
How it works

- Top-level orchestrator breaks goal into tasks
- Delegates to specialist agents
- Each specialist may have sub-tools or sub-agents



Best for: Complex enterprise workflows (e.g., CEO agent → Department agents → Task agents).

Execution: Feedback Loop

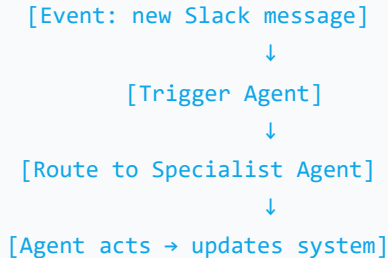


How it works

- Output goes to a critic agent that scores it
- Sends back for revision if score is low
- Loop runs until quality threshold is met or max iterations reached

- ✓ **Best for:** Quality-critical outputs (e.g., Writer → Editor → Score < 8 → back to Writer).

Execution: Event-Driven



How it works

- Agents don't run continuously — they activate on events
- Most cost-efficient architecture for production systems

-  **Best for:** Automations and ops (e.g., New ticket → triage → specialist → resolve).

Human in the Loop

Fully autonomous agents fail on high-stakes decisions. Human-in-the-loop is not a weakness — it's architecture.

🔴 Scenario: Procurement Agent

Task: Order server infrastructure for Q3 scaling (₹50 lakhs budget)

✓ Researches vendors & prepares order

⏸ **Pauses** before placing ₹50 lakh order to ask for human approval

🔵 3 HITL Patterns

→ **Approve before action:** Agent pauses, human approves

→ **Review before output:** Human reviews final task before delivery

→ **Interrupt anytime:** Human can stop or redirect mid-execution



Always use HITL for irreversible actions, payments, and external comms.